

Verification and Validation Report: Software Engineering

Team #16, The Chill Guys

Hamzah Rawasia

Sarim Zia

Aidan Froggatt

Swesan Pathmanathan

Burhanuddin Kharodawala

March 8, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

symbol	description
T	Test

[symbols, abbreviations or acronyms – you can reference the SRS tables if needed —SS]

Contents

1	Revision History	i
2	Symbols, Abbreviations and Acronyms	ii
3	Functional Requirements Evaluation	1
3.1	User Verification	1
3.2	Ride Matching	1
4	Nonfunctional Requirements Evaluation	2
4.1	Usability	2
4.2	Performance	2
4.3	etc.	2
5	Comparison to Existing Implementation	2
6	Unit Testing	2
7	Changes Due to Testing	2
8	Automated Testing	2
9	Trace to Requirements	3
10	Trace to Modules	3
11	Code Coverage Metrics	4

List of Tables

1	Unit test to module traceability. Columns are M1 (Authentication & Verification), M2 (User Profile), M3 (Route & Trip), M4 (Matching), M6 (Notification), M7 (Rating & Feedback), M8 (Safety & Reporting), M9 (Payment & Cost Estimation), M11 (Pricing), M12 (Database).	4
---	---	---

List of Figures

This document ...

3 Functional Requirements Evaluation

3.1 User Verification

1. test-FR1-1: Initial State: No existing account in the system.
Input: Valid McMaster email, password, and driver license upload (for drivers).
Expected Output: Verification email sent; account activated after confirmation.
Actual Output: Verification email sent; account activated after confirmation.
Result: Pass
2. test-FR1-2: Initial State: No existing account in the system.
Input: Non-McMaster email or invalid license format.
Expected Output: Account creation rejected with an error message displayed to the user.
Actual Output: Account creation rejected with an error message displayed to the user.
Result: Pass

3.2 Ride Matching

1. test-FR2-1: Initial State: Verified users with valid profiles and schedules exist in the system.
Input: A rider and driver with overlapping routes and times.
Expected Output: Match proposal generated with a computed cost share presented to both users.
Actual Output: Match proposal generated with a computed cost share presented to both users.
Result: Pass
2. test-FR2-2: Initial State: Verified users with valid profiles and schedules exist in the system. Input: A rider and driver with non-overlapping schedules.
Expected Output: “No match found” message returned to the user.

Actual Output: “No match found” message returned to the user.
Result: Pass

4 Nonfunctional Requirements Evaluation

4.1 Usability

4.2 Performance

4.3 etc.

5 Comparison to Existing Implementation

This section will not be appropriate for every project.

6 Unit Testing

7 Changes Due to Testing

[This section should highlight how feedback from the users and from the supervisor (when one exists) shaped the final product. In particular the feedback from the Rev 0 demo to the supervisor (or to potential users) should be highlighted. —SS]

8 Automated Testing

The Hitchly project implements automated testing as planned in the VnV Plan Section 3.6 (Automated Testing and Verification Tools) and Section 5 (Unit Test Description). The implementation uses Vitest as the primary testing framework (the plan referenced Jest; the codebase adopted Vitest, which is Jest-compatible). Unit tests live in the backend and shared packages: under `apps/api/` in `trpc/routers/__tests__` and `services/__tests__`, and in `packages/db/tests/`, `packages/emails/`, and `packages/utils/`. They cover business logic for authentication and verification, user profile, route

and trip, matchmaking, notifications, rating and feedback, safety and reporting, payment and cost estimation, pricing, and the database schema; external dependencies (database, Stripe SDK, Google Maps API) are mocked. Static analysis and formatting are enforced with ESLint and Prettier; the CI pipeline runs `pnpm lint` and `pnpm format:check` on every push and pull request.

Continuous integration is implemented in GitHub Actions; the workflow file is `.github/workflows/ci.yml`. On code changes the workflow runs type-check, lint, format-check, build, and test; the test job uses a Postgres service container and `pnpm test` to run the Vitest suites across the monorepo. Code coverage is supported by Vitest’s integrated reporter (e.g. `vitest --coverage`). The test IDs used elsewhere in this report (including Section 10, Trace to Modules) match the unit test identifiers in the VnV Plan Section 5, providing clear traceability between the plan and the implemented automated testing.

9 Trace to Requirements

10 Trace to Modules

The following table maps unit test IDs from the VnV Plan Section 5 to the modules defined in the Module Guide (design document, Section 5). Only modules with unit tests in the plan are shown; M5 (Scheduling) and M10 (Admin & Moderation) are omitted as they have no dedicated unit tests and are covered by system-level testing. An “X” indicates that the test(s) in that row exercise the corresponding module.

Test ID	M1	M2	M3	M4	M6	M7	M8	M9	M11	M12
test-ut-auth-1 to test-ut-auth-3	X									
test-ut-profile-1 to test-ut-profile-10		X								
test-ut-trip-1 to test-ut-trip-34			X							
test-ut-match-1 to test-ut-match-8				X						
test-ut-notif-1					X					
test-ut-review-1, test-ut-review-2						X				
test-ut-complaint-1							X			
test-ut-payment-1 to test-ut-payment-20								X		
test-ut-pricing-1, test-ut-pricing-2									X	
test-ut-schema-1, test-ut-schema-2										X

Table 1: Unit test to module traceability. Columns are M1 (Authentication & Verification), M2 (User Profile), M3 (Route & Trip), M4 (Matching), M6 (Notification), M7 (Rating & Feedback), M8 (Safety & Reporting), M9 (Payment & Cost Estimation), M11 (Pricing), M12 (Database).

The full list of individual test case IDs and their descriptions is given in the VnV Plan Section 5.

11 Code Coverage Metrics

References

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?
4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)